

FastAPI

Complete Question & Answer Guide

Covers: Core Concepts • Pydantic • Databases • Routers • Auth • Testing

| May 2026

Section 1 — Core Concepts

What FastAPI is, how it works, and why it is used

Q What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of Starlette (for async web handling) and Pydantic (for data validation). It uses standard Python type hints to define request and response shapes, which enables automatic validation, serialization, and documentation.

Q Why is FastAPI faster than Flask and Django?

Flask and Django are built on WSGI (Web Server Gateway Interface), which is synchronous — it handles one request at a time per worker. FastAPI is built on ASGI (Asynchronous Server Gateway Interface), which supports async/await natively. This means FastAPI can handle many concurrent requests without blocking, making it significantly faster for I/O-heavy workloads like calling external APIs or databases — which is exactly what AI applications do.

Q What is ASGI and how does it differ from WSGI?

WSGI (Synchronous): handles one request at a time per worker. The server waits for each request to complete before starting the next.

ASGI (Asynchronous): can handle many requests concurrently. While waiting for a database or API response, it switches to another request instead of sitting idle.

For AI APIs that constantly call OpenAI, vector databases, or embeddings services, ASGI is dramatically more efficient.

Q What is uvicorn?

Uvicorn is an ASGI server — it is the program that actually runs your FastAPI app and listens for incoming HTTP requests. You run it with:

```
uvicorn main:app --reload
```

Here 'main' is the filename, 'app' is the FastAPI instance, and '--reload' auto-restarts on code changes during development.

```
uvicorn main:app --reload
```

Q What is the /docs page in FastAPI?

FastAPI automatically generates an interactive API documentation page at /docs (Swagger UI) and /redoc. It reads all your route definitions, Pydantic models, and docstrings to create this documentation automatically — no extra work needed. You can execute real API calls directly from the browser, which is extremely useful for testing and development.

Q What is a decorator in FastAPI and what does `@app.get('/')` do?

A decorator is a Python feature that wraps a function and adds behaviour to it. In FastAPI, `@app.get('/')` tells FastAPI two things at once:

1. The URL path to listen on ('/')
2. The HTTP method to accept (GET)

When a GET request arrives at '/', FastAPI runs the decorated function and returns its result as a JSON response.

```
@app.get('/')
def welcome():
    return {'message': 'Hello!'}
```

Q What is the difference between a path parameter and a query parameter?

Path parameter: part of the URL path itself, defined inside curly braces {}. It is always required.

Example: `/api/cars/5` → `id=5`

Query parameter: appended after ? in the URL, defined as a function argument without {}. It is typically optional.

Example: `/api/cars?size=m&doors=3`

FastAPI determines which is which automatically — if the argument name matches a {} in the URL it is a path parameter; otherwise it is a query parameter.

```
# Path parameter
@app.get('/api/cars/{id}')
def car_by_id(id: int): ...

# Query parameter
@app.get('/api/cars')
def get_cars(size: str | None = None): ...
```

Q What happens when you pass the wrong type to a path parameter?

FastAPI validates the type at the boundary. If you declare `id: int` and someone calls `/api/cars/hello`, FastAPI returns a 422 Unprocessable Entity error automatically — your function never runs. However, `str` accepts anything since everything in a URL is a string by nature, so `/api/cars/123` with `id: str` works fine.

Q What is `async def` and what is the difference between concurrency and parallelism?

`async def` defines an asynchronous function that can use 'await' to pause and let other tasks run while waiting.

Concurrency: one thread juggling multiple tasks by switching between them during I/O waits. FastAPI uses this model — while waiting for OpenAI to respond it serves another user.

Parallelism: multiple CPU cores doing different tasks simultaneously at the exact same moment.

For AI APIs, most waiting is I/O waiting (database, LLM, embeddings). Async/await handles this efficiently through concurrency, not parallelism.

```
async def get_answer(question: str):  
    result = await openai_client.chat(question) # non-blocking  
    return result
```

Section 2 — Pydantic & Data Validation

How FastAPI validates, converts, and documents data

Q What is Pydantic and why does FastAPI use it?

Pydantic is a Python library for data validation using type hints. Python does not enforce types at runtime by default — Pydantic does. FastAPI uses Pydantic models to:

1. Validate incoming request data (wrong types return 422 automatically)
2. Convert/coerce data types (e.g. '5' string → 5 integer)
3. Serialize response data to JSON
4. Auto-generate API documentation in /docs

You define the shape once and FastAPI handles everything else.

Q What is BaseModel and how do you use it?

BaseModel is Pydantic's base class. Any class that inherits from it becomes a validated data model. FastAPI uses it for request bodies — when a POST endpoint declares a parameter of type BaseModel, FastAPI automatically reads the JSON body, validates it, and passes the object to your function.

```
from pydantic import BaseModel

class CarInput(BaseModel):
    size: str
    doors: int
    fuel: str | None = 'electric' # optional with default

@app.post('/api/cars')
def add_car(car: CarInput): # FastAPI validates automatically
    return car
```

Q What does model_validate() do?

model_validate() converts a plain Python dictionary into a Pydantic model object. This is commonly used when loading data from a JSON file or database result that comes back as a plain dict, and you need it as a typed, validated object.

```
data = {'size': 'm', 'doors': 4, 'fuel': 'electric', 'id': 1}
car = CarOutput.model_validate(data)
print(car.size) # 'm' — accessed with dot notation, not car['size']
```

Q What does model_dump() do?

model_dump() converts a Pydantic model object back into a plain Python dictionary. This is required before saving to JSON files or passing to systems that do not understand Pydantic objects — like json.dump().

```
car = CarOutput(id=1, size='m', doors=4, fuel='electric')
data = car.model_dump()
# → {'id': 1, 'size': 'm', 'doors': 4, 'fuel': 'electric'}
```

Q What is the difference between CarInput and CarOutput?

CarInput: what the user sends when creating a resource. It does NOT include id because the system assigns the id — the user should not control it.

CarOutput: what the API returns after creation or retrieval. It includes the id assigned by the database.

CarOutput inherits from CarInput so it automatically has all its fields — plus adds id. This separation gives you clean control over what goes in vs what comes out.

```
class CarInput(BaseModel):
    size: str
    doors: int

class CarOutput(CarInput): # inherits size + doors
    id: int                 # adds id
```

Q Why use str | None = None for optional parameters?

This means the parameter is truly optional:

- str | None declares the type can be a string OR None
- = None sets the default value to None if not provided

Without = None, FastAPI would still require the parameter even if the type allows None. Both parts are needed together to make a parameter genuinely optional.

```
def get_cars(size: str | None = None): # fully optional
    if size:
        # filter by size
    pass
```

Section 3 — HTTP Methods, CRUD & Status Codes

How APIs communicate using HTTP standards

Q What is CRUD and which HTTP method maps to each operation?

CRUD is the standard set of operations for any data-driven application:

Create → POST (add new data)
Read → GET (retrieve data)
Update → PUT (replace existing data)
Delete → DELETE (remove data)

Every REST API is built around these four operations.

Q What are the most important HTTP status codes?

200 OK — request succeeded, response contains data
201 Created — POST succeeded, new resource created
204 No Content — request succeeded, no data to return (used for DELETE)
400 Bad Request — client sent invalid data
401 Unauthorized — not authenticated (no or invalid token)
403 Forbidden — authenticated but not allowed
404 Not Found — resource does not exist
422 Unprocessable — validation error (wrong data types)
500 Server Error — something crashed on the server

Q What is HTTPException and why is it better than returning an error dict?

HTTPException sends a proper HTTP error response with the correct status code. Returning `{'error': 'not found'}` sends a 200 OK status — which is misleading and wrong.

HTTP status codes are a universal language. Any client (browser, mobile app, another service) automatically understands a 404 without reading the response body. Proper error handling is essential in production APIs.

```
# Wrong — returns 200 OK even though it's an error
return {'error': 'Car not found'}

# Correct — returns proper 404 status
raise HTTPException(status_code=404, detail='Car not found')
```

Q What does status_code=204 mean on a DELETE endpoint?

204 means 'success with no content.' When you delete a resource there is nothing to return — the resource is gone. Sending an empty response body with 204 is the standard HTTP pattern for successful deletions. This is why DELETE endpoints return None.

```
@app.delete('/api/cars/{id}', status_code=204)
def remove_car(id: int) -> None:
```

```
# delete logic
...
```

Q What is a list comprehension and how is it used for filtering?

A list comprehension is a concise Python pattern to create a new list by iterating and optionally filtering another list. It follows the pattern: [expression for item in iterable if condition]

```
# Filter cars by size
result = [car for car in db if car.size == 'm']

# Filter cars with at least 3 doors
result = [car for car in db if car.doors >= 3]

# Same as a for loop but in one line
```


Section 4 — Database & SQLAlchemy

Persistent storage with SQLite, SQLAlchemy, and sessions

Q What is SQLAlchemy and why use it with FastAPI?

SQLAlchemy is a library that combines Pydantic and SQLAlchemy into one. It lets a single class act as both a Pydantic validation model AND a database table definition. Without SQLAlchemy you would need separate classes for each. It was created by the same author as FastAPI, so they work seamlessly together.

Q What does table=True do in a SQLAlchemy class?

table=True tells SQLAlchemy that this class represents an actual database table. Classes without table=True are pure Pydantic models used only for validation. Classes with table=True are both a validation model and a database table.

```
class CarInput(SQLAlchemy):          # Pydantic model only — no table
    size: str
    doors: int

class Car(CarInput, table=True):     # creates 'car' table in DB
    id: int | None = Field(default=None, primary_key=True)
```

Q What is a primary key and why is id set to None by default?

A primary key is the unique identifier for each row in a database table. No two rows can share the same primary key.

The id is set to None by default because when you create a new record you do not know the id yet — the database assigns it automatically using auto-increment. You pass None and the database fills it in after the INSERT.

```
class Car(CarInput, table=True):
    id: int | None = Field(default=None, primary_key=True)
    # id is None before saving, then DB assigns 1, 2, 3...
```

Q What is a foreign key and what relationship does it create?

A foreign key is a column that stores the primary key of a row from another table. It creates a link between two tables — the 'belongs to' relationship.

Example: car_id in the Trip table stores the id of the car that trip belongs to. A trip with car_id=3 belongs to the car with id=3.

```
class Trip(TripInput, table=True):
    car_id: int | None = Field(default=None, foreign_key='car.id')
    # This trip belongs to whatever car has this id
```

Q What is Relationship() in SQLAlchemy and how is it different from a foreign key?

Foreign key (`car_id`): a real database column that stores a number linking to another table.

`Relationship()`: a Python-only shortcut — no database column. It lets you navigate between related objects using dot notation.

They work together: `car_id` stores the link in the database, `Relationship()` makes it easy to access in Python.

```
class Car(CarInput, table=True):
    trips: list[Trip] = Relationship(back_populates='car')
    # No column in DB — just Python convenience

# Usage:
car.trips      # → list of all Trip objects for this car
trip.car       # → the Car object this trip belongs to
```

Q What is a database session and what does `get_session()` do?

A session is a temporary connection to the database for a single request. It tracks changes (adds, updates, deletes) and commits them together.

`get_session()` opens a session, yields it to the endpoint, then automatically closes it when the endpoint finishes — even if it crashes. This ensures connections are never left open.

```
def get_session():
    with Session(engine) as session:
        yield session    # pause, hand session to endpoint
        # session closes automatically when endpoint finishes
```

Q What is the difference between `session.commit()` and `session.refresh()`?

`session.commit()`: actually writes all staged changes to the database. Until `commit()` is called, changes exist only in memory.

`session.refresh(obj)`: re-reads the object from the database to sync it with the latest database values. Used after adding a new record because the database assigns an id that the Python object does not yet know about.

After `add + commit`, the object still shows `id=None`. After `refresh`, it shows the real id assigned by the database.

```
session.add(new_car)      # stage
session.commit()          # write to DB — DB assigns id=7
print(new_car.id)         # still None!
session.refresh(new_car)  # sync from DB
print(new_car.id)         # now 7 ☐
```

Q What is dependency injection and what does `Depends()` do?

Dependency injection means FastAPI automatically calls a function and passes its result into your endpoint — you do not call it yourself.

`Depends(get_session)` tells FastAPI: 'before running this endpoint, call `get_session()` and inject what it

yields as the session parameter.' FastAPI manages the full lifecycle — opening, injecting, and closing the session automatically.

```
@app.get('/api/cars')
def get_cars(
    session: Annotated[Session, Depends(get_session)], # injected automatically
    size: str | None = None
):
    ...
```

Q What does yield do in a dependency function vs return?

return: the function completes immediately and is done.

yield: the function pauses at the yield point, hands the value to the endpoint, waits for the endpoint to finish, then resumes for cleanup. Everything after yield is cleanup code.

This is why `get_session()` uses `yield` — it needs to close the database connection **AFTER** the endpoint finishes, not before.

```
def get_session():
    with Session(engine) as session:
        yield session # pause here, endpoint runs
    # resumed here after endpoint finishes — session closes
```

Section 5 — Routers & Project Structure

Organising a production FastAPI project

Q What is an APIRouter and why use it instead of putting everything in main.py?

APIRouter is a mini FastAPI app for a specific feature. It lets you split endpoints across multiple files — one file per feature (cars, users, auth, payments).

In production, a single main.py with all endpoints becomes unmanageable. Routers keep code organised, make it easier to find bugs, and let multiple developers work on different features without conflicts.

```
# routers/cars.py
router = APIRouter(prefix='/api/cars')

@router.get('/')      # registers GET /api/cars/
def get_cars(): ...

# main.py
app.include_router(cars_router) # plug in the router
```

Q What does the prefix parameter do in APIRouter?

prefix is a URL prefix that gets added to every route in that router automatically. Instead of writing '/api/cars' on every endpoint, you set it once on the router and write just '/' and '{id}'.

Changing the prefix in one place updates all endpoints at once — much easier to maintain.

```
router = APIRouter(prefix='/api/cars')

@router.get('/')      # → GET /api/cars/
@router.get('/{id}')  # → GET /api/cars/5
@router.post('/')     # → POST /api/cars/
```

Q What is __init__.py and why is it needed in the routers/ folder?

__init__.py is an empty file that marks a folder as a Python package. Without it, Python does not recognise the folder as importable and 'from routers.cars import router' would fail with a ModuleNotFoundError. You do not put any code in it — just create it empty.

Section 6 — Authentication & Security

Password hashing, OAuth2 tokens, and protected endpoints

Q Why should you never store passwords as plain text?

If your database is ever compromised, all user passwords are immediately exposed. Since many users reuse passwords across services, this puts their other accounts at risk too.

Instead, store a hash — a one-way mathematical transformation of the password. Even if hackers steal the database, they cannot reverse hashes back to the original passwords.

```
# NEVER do this:
user.password = 'hello123'           # plain text in DB — dangerous

# Always do this:
user.hashed_password = bcrypt.hash('hello123')  # irreversible hash
```

Q What is bcrypt and how does password verification work?

bcrypt is a widely-used password hashing algorithm. It is deliberately slow (to resist brute-force attacks) and adds a random salt to each hash (so two identical passwords produce different hashes).

Verification works by hashing the input again and comparing the two hashes — you never unhash. If the hashes match, the password is correct.

```
pwd_context = CryptContext(schemes=['bcrypt'])

# Hash when registering:
hashed = pwd_context.hash('hello123')

# Verify when logging in:
pwd_context.verify('hello123', hashed)  # True ☐
pwd_context.verify('wrongpass', hashed) # False ☐
```

Q What is OAuth2 Password Flow?

OAuth2 is a standard authentication protocol. The Password Flow works in two steps:

Step 1 — Login: user sends username + password to POST /auth/token. Server returns a token.

Step 2 — Every request: user includes the token in the Authorization header. Server validates the token and identifies the user.

This is like a concert wristband — show your ID once at the entrance (login), use the wristband (token) at every door inside.

```
# Step 1: Login
POST /auth/token
Body: username=gurleen&password=hello123
Response: {'access_token': 'gurleen', 'token_type': 'bearer'}

# Step 2: Use token
GET /api/cars
```

Header: Authorization: Bearer gurleen

Q What is the difference between User, UserInput, and UserOutput?

UserInput: what the client sends during registration — username and plain text password. Never stored directly.

User (table=True): the actual database record — stores username and hashed_password. Never sent to clients.

UserOutput: what the API returns about a user — only id and username. No password or hash is ever exposed outside the server.

Q How do you protect an endpoint in FastAPI so only authenticated users can access it?

Add `get_current_user` as a dependency. FastAPI runs it before your endpoint. If the token is valid, the endpoint runs. If the token is missing or invalid, FastAPI returns 401 Unauthorized automatically — your endpoint code never executes.

```
@router.delete("/{id}", status_code=204)
def remove_car(
    id: int,
    session: Annotated[Session, Depends(get_session)],
    current_user: UserOutput = Depends(get_current_user)  # ← gate
) -> None:
    # only runs if token is valid
    ...
```

Q What does `app.dependency_overrides` do?

`dependency_overrides` lets you replace a dependency with a different one at runtime. In testing, you use it to swap the real database session with a test database session — without changing any endpoint code. The endpoints do not know or care which session they receive.

```
# Replace real DB with test DB for all tests
app.dependency_overrides[get_session] = get_test_session

# All endpoints now use the test database automatically
```

Section 7 — Testing with Pytest

Automated testing for reliable, production-ready APIs

Q What is pytest and why is automated testing important?

Pytest is Python's standard testing library. You write functions that call your API and assert the responses are correct. Pytest runs them all automatically and reports pass/fail.

Automated testing is important because:

- Manual testing 50+ endpoints after every code change is not feasible
- Tests catch regressions — bugs introduced by new changes
- In AI engineering, tests validate LLM output quality (accuracy, faithfulness, relevance)

Q What is TestClient and how does it work?

TestClient is a FastAPI tool that simulates HTTP requests to your app in memory — no actual server needed. It calls your app directly, making tests fast and isolated. You import your app and wrap it with TestClient.

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

# Call your app like a real HTTP client
response = client.get('/api/cars')
response = client.post('/api/cars', json={...})
```

Q Why use a separate test database instead of the real database?

Testing against the real database would add, modify, and delete real data. A test run could corrupt your production data. A separate test database (typically an in-memory SQLite) gets created fresh for tests and discarded afterwards — your real data is never touched.

Q What is the Arrange-Act-Assert pattern?

The standard structure for every test:

Arrange: set up the data and conditions for the test.

Act: call the API or function being tested.

Assert: verify the result is what you expected.

This makes tests easy to read and understand.

```
def test_add_car():
    # Arrange
    new_car = {'size': 'm', 'doors': 4, 'fuel': 'electric'}

    # Act
    response = client.post('/api/cars', json=new_car)
```

```
# Assert
assert response.status_code == 200
assert response.json()['size'] == 'm'
assert response.json()['id'] is not None
```

Q Why must test functions start with test_?

Pytest discovers tests by scanning files for functions whose names start with test_. Functions with any other name are ignored. This convention also makes it immediately obvious to any developer reading the code that a function is a test.

```
def test_get_cars():      # ☐ pytest finds and runs this
def get_cars_test():     # ☐ pytest ignores this
def check_cars():        # ☐ pytest ignores this
```

Q How do you run tests and what does the -v flag do?

Run all tests in a folder with pytest. The -v (verbose) flag shows each individual test name and its pass/fail status. Without -v you only see a summary.

```
pytest test/ -v

# Output:
# test/test_cars.py::test_get_cars_empty PASSED ☐
# test/test_cars.py::test_add_car PASSED ☐
# test/test_cars.py::test_delete_car FAILED ☐
```